Stored Crop Quality Project Walkthrough

Simple explanation, technical details, current results, and tomorrow's demo

Prepared 1 November 2026

The main message

A runnable research demonstration is built and trained. Its main five-crop results come from simulated data, so they show that the software can learn the simulator's patterns, not that the model is accurate on real crops.

1. The project in one minute

The research idea is to read storage conditions and crop-emitted gas-sensor responses from the same crop lot at the same time. The system estimates the crop's current quality class and its risk of becoming unacceptable at the next inspection.

The target crops are wheat, maize, table potato, cured dry onion, and spinach. The desired hardware pairing is AeroSense environment readings plus nine Smart Nose response channels: MQ2, MQ3, MQ4, MQ5, MQ6, MQ7, MQ8, MQ9, and MQ135.

What is complete

The data pipeline, grouped training/evaluation, saved classifiers and survival models, notebooks, command-line tools, and local browser demo are present. A second synthetic dataset (v2) was generated and trained separately.

What is not complete

There is still no measured dataset for these crops that pairs all requested sensor channels with independent quality inspections and failure follow-up. Therefore no real five-crop spoilage or shelf-life accuracy has been demonstrated.

2. Explain it like I am new to the topic

Imagine putting a box of spinach in storage. A thermometer and humidity sensor tell us how hot and damp the storage is. A group of MQ sensors gives a pattern of electrical responses to gases around the crop. At check-up time, a person also inspects the spinach and writes down whether it is acceptable, warning, or spoiled.

The computer learns from many examples where the sensors and the inspector looked at the same crop lot at matching times. Later, when it sees new readings, it estimates the current class and the chance of a bad result by the next check.

In our current main dataset, the computer examples were generated by a simulator. They look like paired measurements in a file, but no person measured those crop lots with the real device. The demo is real software; its sensor readings and answers are simulated.

A short explanation you can say to faculty

Suggested 30-second version

I built a leak-aware pipeline for crop quality classification and time-to-first-unacceptable-inspection. It compares environment-only, nine-MQ-only, and combined inputs, and keeps all observations from a starting lot together in the test split. The five-crop data and labels are simulated, so the scores are software-development results only. Public real datasets have different crops, sensors, or targets; I keep them separate. The next validation step is collecting synchronized sensor readings and independent inspections from real lots.

3. What is a lot, a path, and a reading

A lot is the physical group of produce that a study follows, such as one labeled sack of wheat or one storage bin of potatoes. A path is the storage condition history assigned to that lot. A reading is one time point along that history.

Word	Simple meaning	Project field / example
Lot	The crop group we want to follow	`sim_lot_id`
Path / trajectory	The storage history the lot experiences	`sim_trajectory_id`, `scenario_SIM`
Reading	One synchronized sensor and inspection time	`storage_age_days` plus fields on the same row
Inspection	A human or lab assessment of crop quality	`inspection_defect_pct_SIM`, `status_SIM` in simulation
Censored follow-up	Monitoring ended while still acceptable; failure time is unknown	event=0 and follow-up time recorded

The two simulated datasets

Dataset	Contents	How to interpret it
v1 supplied bundle	120,000 rows; 500 starting lots (100 per crop); 6 paths per lot; 40 readings per path; 3,000 trajectories.	Each lot branches into six simulated scenarios. Rows and paths are repeated simulator observations, not independent real crop samples.
v2 generated here	20,000 rows; 500 generated lots (100 per crop); 1 path per lot; 40 readings each.	No copied counterfactual branches. Still fabricated; one synthetic ID is not a biological replicate.

The v1 count equation is 5 crops x 100 simulated lots x 6 paths x 40 readings = 120,000 rows. The v2 equation is 5 x 100 x 1 x 40 = 20,000 rows. A row count is not the same as the number of independent lots.

In both versions, a CSV row contains environment and MQ values generated for that simulated lot and time. This is an internally paired simulation. It must not be confused with real device readings from a real lot.

Other available real sources stay separate

Source	What it has	Why it is not the five-crop combined dataset
Zenodo 15469472 tomato	A small table with 12 MQ-family Rs/R0 channels, weight/color, storage day and scenario; separate repeated tomato records.	Target is storage scenario, not inspected spoiled status. No AeroSense temperature/RH; sensor set is not the exact nine channels.
Zenodo 17285312 nine MQ	The nine named MQ channels with food/day recording files at roughly room temperature.	No independent acceptable/warning/spoiled inspections or measured failure time.
Mendeley potato storage	Environmental temperature/RH/core temperature/CO2 histories.	No MQ channels and no verified row-level quality join.
TriModal Ripeness 6 lead	A public, large tomato/produce dataset with MQ-4 and binary visible-spoilage organization.	Not downloaded or audited yet; MQ-4 only, no AeroSense fields or full nine channels. Fruit-ID matching must be checked.

Public source links: Zenodo 15469472 | Zenodo 17285312 | Potato Mendeley record | TriModal dataset record

4. How the faculty sensor requirements map to the CSV

The mapping below describes the intended device system and the similarly named columns in the simulator. In v1 and v2, those values are generated. They are not readings from the faculty's AeroSense or Smart Nose hardware.

Faculty requirement	CSV fields	What it means / caution
Crop identity	`crop`, `crop_group`	Crop category; vegetables have no grain-moisture value in these data.
Same lot and same time	`sim_lot_id`, `sim_trajectory_id`, `storage_age_days`	Grouping/join keys. In simulation the channels share a generated row. For a real study, create actual stable lot/sample IDs and synchronized timestamps.
AeroSense current environment	`Temperature(°C)`, `Humidity(%)`, `Pressure(hPa)`, `AQI`, `Gas (kOhms)`, `Altitude(m)`, `DewPoint(°C)`, `HeatIndex(°C)`	Device-named environmental fields. AQI and gas values are supplied as fields; project does not need to recompute firmware outputs. Simulated values do not reproduce firmware.
Nine Smart Nose channels	`MQ2_response_index_SIM`, `MQ3_response_index_SIM`, `MQ4_response_index_SIM`, `MQ5_response_index_SIM`, `MQ6_response_index_SIM`, `MQ7_response_index_SIM`, `MQ8_response_index_SIM`, `MQ9_response_index_SIM`, `MQ135_response_index_SIM`	Relative, cross-sensitive response indices. Do not call these nine exact gas concentrations.
Crop/lot context	`grain_moisture_pct_SIM`, `lot_mass_kg_SIM`, optional `initial_inspection_defect_pct_SIM`	Synthetic context; grain moisture is missing for potato, onion, and spinach. Initial inspection is usable only if measured in a real deployment.
Past/current history	`past_mean_temp_C_SIM`, `past_mean_rh_pct_SIM`, `past_hot_degree_days_SIM`	Summaries through the current reading. They do not include future measurements in the generator.
Quality inspection now	`inspection_defect_pct_SIM`, `inspection_defect_type_SIM`, target `status_SIM`	Generated labels. Defect columns are outcomes/inspection evidence, never predictors for the sensor-only status model.
Failure and censoring	`failure_interval_start_days_SIM`, `failure_interval_end_days_SIM`, `event_observed_SIM`, `observed_followup_days_SIM`, `remaining_days_to_first_failed_check_SIM`, `post_failure_SIM`	Survival targets/bookkeeping only; excluded from predictor inputs.
Scenario/time/provenance	`scenario_SIM`, `sim_timestamp_utc`, `Date`, `Time`, `data_origin`	Audit/split context. Scenario IDs and fictional absolute timestamps are not predictors.

How the simulated label is made

The v1 and v2 status classes use the simulator's defect-percentage rule: acceptable below 7%, warning from 7% to below 15%, and spoiled at 15% or higher. These are assumptions used to make a three-class task. They are not approved quality limits for wheat, maize, potato, onion, or spinach. A real study needs crop-specific inspection standards and a written rubric agreed with the advisor.

Important target relationship

The simulator creates both sensor responses and inspection labels from a shared synthetic deterioration process. A model can score well by recovering that built-in relationship. This is not an independent real-world test of whether the hardware predicts spoilage.

What is excluded to prevent leakage

Predictors do not include the answer (`status_SIM`), inspection outcomes, failure intervals, event/censor flag, follow-up/remaining days, post-failure flag, lot/path IDs, scenario labels, or fictional dates. Those fields are used to create targets, split or audit the data, or describe the outcome. A predictor must only use values known at the prediction time.

The optional initial-inspection feature is compared separately because it is only deployable if someone actually collects that inspection at the start. Preprocessing imputers are fitted inside training pipelines, not on the held-out test set.

5. How training and testing works

The unit kept together during splitting is the starting lot (`sim_lot_id`). Every reading and every trajectory belonging to a v1 starting lot stays on one side of the split. In v2, each lot has one path, but its 40 readings still stay together.

Stage	What happens	Why it matters
Training	Models learn from 400 lots; preprocessing and model selection use training data only.	The model has not seen these lots' outcomes during fitting.
Held-out testing	Final metrics use 100 other complete lots (20 lots per crop).	Closer to the question: how does it perform on a new lot from the same simulator?
Grouped validation	Whole lots are held out in validation too; XGBoost stopping rounds are chosen inside training.	Prevents 40 repeated observations from the same simulated lot leaking across folds.
Final saved model	Fitted pipelines and metrics are written under `artifacts/` (v1) or `artifacts_v2/` (v2).	Demo loads saved models; it does not retrain on page load.

Why compare these input sets

- Environment-only: checks how much the environment-like fields predict in this simulator without MQ signals.
- MQ-only: checks whether the nine simulated response channels contain useful synthetic signal by themselves.
- Combined: checks whether both families add information in the simulator.
- All comparisons use the same held-out lots, so the feature families are compared fairly within this generated dataset.

Why these models

Model	Simple explanation	Role in this project
Majority-class baseline	Always guesses the most common training class.	A minimum bar. A complex model should do better than this simple rule.
Random Forest	Builds many decision trees on different samples/features and averages them.	A strong, practical benchmark for nonlinear tabular sensor data.
XGBoost	Builds trees in sequence; each new tree focuses on patterns the current ensemble still gets wrong.	Captures nonlinear combinations such as crop + age + temperature + sensor response; compare with RF rather than assume it wins.
Discrete-time hazard	At each inspection interval, estimates the chance that the first unacceptable inspection occurs by the next check.	Primary time-varying risk model; can update when a new current reading arrives.
XGBoost AFT	A survival-time comparison model that accepts an event interval or a censored follow-up bound.	Comparator only; event time is not observed exactly between inspections.

Tree ensembles fit this structured table without needing image-like data or a large sequence model. XGBoost is not guaranteed to be superior; the actual held-out evidence decides which model is better for the stated metric. CNNs, LSTMs, and graph models would add complexity without suitable real independent-lot data to justify them.

What the score names mean

Measure	Plain explanation	Why we show it
Macro F1	Calculate F1 for acceptable, warning, and spoiled separately, then average equally.	A good score requires performance across all classes, not just the largest group.
Spoiled recall	Of the truly spoiled readings, how many did the model catch?	Shows how often spoilage is missed. High recall alone can come with many false alarms.
Confusion matrix	A count of true class versus predicted class.	Shows which classes are being confused and whether spoiled items become acceptable predictions.
Log loss	Penalizes confident wrong probabilities more strongly.	Checks whether predicted probabilities are useful, not just the winning class.
Multiclass Brier	Average squared difference between class probabilities and the true one-hot answer.	Another probability-quality measure; lower is better.
AUC	How well scores rank event intervals above non-event intervals.	Discrimination only. It is not days-left accuracy.

6. Results and what they actually show

Never say

The model is 93% accurate on real crop spoilage. The v1 and v2 results are simulator reconstructions, not validated crop accuracy.

Original v1 simulator

The saved original evaluation used 400 training starting lots and 100 held-out starting lots. The combined Random Forest reached macro F1 0.9309; combined XGBoost reached 0.9300. Random Forest's macro F1 was slightly higher, while XGBoost had better log loss (0.1622 vs 0.1783) and Brier score (0.1004 vs 0.1055). Their overall results are close. The simulator's MQ signals are strongly related to its own generated labels, which likely makes the task easier than real deployment.

Feature family	Random Forest macro F1	XGBoost macro F1	XGBoost spoiled recall	XGBoost log loss / Brier
Environment	0.8226	0.8360	0.8445	0.3921 / 0.2382
Nine MQ	0.9230	0.9251	0.9526	0.1738 / 0.1084
Combined	0.9309	0.9300	0.9562	0.1622 / 0.1004

For the combined v1 XGBoost model, per-crop macro F1 / spoiled recall were: wheat 0.9185 / 0.9536; maize 0.9232 / 0.9394; potato 0.9400 / 0.9621; cured dry onion 0.9165 / 0.9391; spinach 0.9506 / 0.9718. These remain simulator-only class results.

New v2 assumption-driven simulator

V2 has one path per generated lot and more explicit lot variation, environmental fluctuation, sensor drift/noise, and censoring. The fixed run contains 20,000 rows and the same 400/100 grouped lot split.

Features	Model	Macro F1	Spoiled recall	Log loss	Brier
Environment	RF	0.8131	0.8270	0.4374	0.2627
Environment	XGBoost	0.8202	0.8449	0.4335	0.2586
Nine MQ	RF	0.7861	0.8103	0.5014	0.3007
Nine MQ	XGBoost	0.8031	0.8376	0.4414	0.2707
Combined	RF	0.8292	0.8677	0.4238	0.2503
Combined	XGBoost	0.8342	0.8800	0.3863	0.2318

Combined XGBoost narrowly leads the listed v2 measures in this run. It is a reasonable model to click through in the v2 demo. This result does not upgrade the evidence to real crop accuracy. The class thresholds and sensor patterns are generated by the same simulation code.

How to explain why v1 and v2 scores differ

The two scores come from different generated datasets, not from the same fixed benchmark. V1's six paths per starting lot create a different task structure than v2's one path per generated lot. The generator's noise, sensor response, and label rules also differ. Do not describe the score change as progress on real crops or compare the decimals as if only the model changed.

Why accuracy is not our only headline

If almost every item were acceptable, a model could guess acceptable every time and look accurate while missing spoiled produce. Macro F1 checks all classes evenly. Spoiled recall specifically reports misses. Log loss and Brier check probability quality. We also inspect per-crop scores and confusion matrices because an overall average can hide a crop that performs poorly.

7. Remaining-life model without inventing failure dates

The event is the first inspection labelled spoiled in the simulation. If one inspection is warning and the next is spoiled, the exact deterioration moment is unknown; the data record a failure interval between those inspections. A trajectory still acceptable when monitoring ends is right-censored: we know it lasted at least through follow-up, but we do not know its eventual failure day.

The discrete-time hazard model makes one prediction per observed interval. It reads the features at the beginning of that interval and predicts whether the first spoiled inspection occurs at the next check. It does not use readings after failure. If a lot is censored, it contributes known event-free intervals only; it does not receive a fabricated event day.

Survival score	v1 simulator	v2 simulator	How to say it
XGBoost hazard interval Brier	0.01545	0.03527	Average squared probability error for next-inspection event rows; lower is better on each simulator.
XGBoost hazard AUC	0.9804	0.8910	Ranks event intervals; not remaining-days accuracy.
AFT interval distance	Median 7.81 d; mean 37.65 d	Median 6.11 d; mean 55.67 d	Observed failures only; prediction compared with an interval, not exact failure time. Pooled across different crop schedules.

Calendar warning

One inspection interval is 7 days for grains, 2 days for potato/onion, and 6 hours for spinach in these generated data. A high hazard AUC does not say the model knows the exact day the crop will spoil. Future-path projections also require explicit future MQ and environment values and are not proof that cooling causes extra shelf life.

8. The working saved-model demonstration

The local web interface loads saved models. To use the v2 models on port 8766, open PowerShell in the project folder and run:

```
.\.venv\Scripts\python.exe -m crop_prototype.web --artifacts artifacts_v2 --port 8766
```

Then open http://127.0.0.1:8766. Use measured crop/environment/MQ input only when it is genuinely paired. The saved example is synthetic and for rehearsal.

The v2 held-out maize example at day 140 returns acceptable 0.7226, warning 0.2524, spoiled 0.0249, and first-spoiled-by-next-7-day-check risk 0.0023. The result is explicitly labelled SIMULATION ONLY. It returns no continuous remaining-days estimate. A probability is the model's conditional estimate under the simulator, not a guarantee.

To rerun that prediction from the saved model without training again:

```
.\.venv\Scripts\python.exe -m crop_prototype.cli predict --input
examples\simulated_v2_prediction_input.json --out artifacts_v2
```

What the interface can and cannot do

- Can enter current crop, age, environment readings, nine MQ responses, history fields, and the next check interval; it returns status probabilities and next-check risk from saved models.
- Can read a properly mapped CSV and report incomplete inputs. It does not invent missing sensor measurements.
- Can accept a separate explicit future path for a conditional simulation. It does not forecast that path, copy today's MQ values forward, or prove a cooling intervention works.
- Does not connect to physical sensors over serial/network. Live sensor integration and hardware calibration are future work.

9. What has been done and what remains

Done in the repository	Still needed for a real research model
Read/audit the original ZIP, README, CSV schema, row/crop/lot/path counts, labels, missingness and ordering.	Collect real synchronized measurements from the same crop lot: AeroSense plus all nine MQ channels, with device IDs, units, calibration and timestamps.
Build environment-only, MQ-only and combined synthetic feature families with label/future leakage exclusions.	Define crop-specific acceptable/warning/unacceptable inspection rubrics with an independent inspector or lab method.
Grouped train/test splits by starting lot; save metrics and fitted pipelines; provide notebooks and CLI.	Collect enough independent lots across harvests, storage units and sensor setups. Hold out entire lots, and preferably a later run/device, for final evaluation.
Build discrete hazard, logistic comparator and XGBoost AFT evaluation with event intervals and censoring.	Follow lots to first unacceptable inspection or record a truthful last-follow-up time for censored lots.
Build a local web demo that loads saved models; make missing fields visible; separate future-path projections.	Validate model transfer, probability calibration, drift, sensor failure, missing channels and performance by crop on held-out real lots.
Review public tomato, nine-MQ and potato storage datasets separately; do not fabricate joins.	Only claim a combined real model after matched records and labels are available and independently evaluated.

Recommended lock-in plan for the next work session

1. Start the v2 browser demo and practice the short explanation. Keep the phrase SIMULATION ONLY visible in the demo and slides.
2. Re-run the focused project checks and inspect the saved JSON metrics against the report. Keep v1 and v2 result tables separate.
3. Prepare the report/presentation around what is demonstrably complete: dataset audit, safe grouped split, model comparisons, censoring-aware workflow, saved-model interface, and honest limits.
4. If time allows, audit the open tomato MQ-4 spoilage archive as a separate task. First verify fruit IDs and gas/label alignment. Do not describe it as nine-MQ + AeroSense or mix it with potato/grain data.
5. End with the real-data pilot protocol: crop/cultivar, lot definition, sensor matching tolerance, inspection rubric, measurement schedule, event/censor rule, and independent holdout plan.

10. Questions faculty may ask

Question	Clear answer
Is this a real-crop validated model?	No. The main five-crop labels and readings are simulated. The app works; real-world predictive accuracy is not established.
Why are there 120,000 rows but only 500 lots?	Each v1 starting lot was simulated once and branched into six paths, then read 40 times per path. Rows are repeated observations, not 120,000 independent samples.
Why split by lot?	All measurements and paths from one starting lot are related. Keeping them together prevents related data from appearing in both training and test.
Why XGBoost?	It is a strong tree-boosting option for nonlinear tabular features. We compare it with Random Forest and a majority baseline; we do not assume it must win.
Why use macro F1 and spoiled recall?	Macro F1 gives each quality class equal weight. Spoiled recall measures how many spoiled examples are caught. We also show probability losses because decisions may use probabilities.
Can it tell exact days remaining?	Not from today's input alone. Survival estimates risk over observed inspection intervals; the AFT comparison uses censored/interval data but is still simulator-only.
Can we combine the separate tomato MQ and potato environment files?	No. Their rows are from different studies and cannot be asserted to be the same lot/time. Combining them would invent paired measurements.
What is the next real experiment?	Collect synchronized environment + MQ readings and independent inspections from identified lots, then hold out entire lots and a separate run for evaluation.

Reference files in the project

- README.md - setup and commands
- RESULTS.md - v1 and public tomato study results
- SIMULATION_V2.md - v2 assumptions, metrics, and saved prediction
- DATASET_REVIEW.md - source-by-source data audit
- DEMO_GUIDE.md - short demonstration notes
- PAPER_DRAFT.md - manuscript draft and limitations